

Operating Systems - Process Management

What a process is

A process is a program in execution. The program itself is only a passive list of instructions sitting on disk, while a process is the active thing the operating system schedules. One program can therefore correspond to several processes at the same time, and a process can create children that run the same program. To the operating system a process is an address space, an execution context holding the register values, and a set of open resources. The execution context is kept in a data structure called the process control block.

The process control block

The process control block is the record the operating system keeps for a single process. Because the operating system must be able to stop a process and resume it later, the process control block must contain everything needed to reconstruct the process exactly. A typical process control block holds the process identifier, the program counter, the processor register values, the stack pointer, memory management tables for the address space, and the scheduling state of the process.

The program counter field matters most for the context switch described later. The program counter records the address of the next instruction to execute, so restoring it tells the processor where to resume. The register values in the process control block, such as the stack pointer, must also be restored for the process to continue.

The five process states

Every process passes through a small set of states. The New state is the state a process is in after it is created but before the operating system has admitted it to the set of processes that may run. A process in the New state still needs its address space prepared and its process control block initialised.

A process in the Ready state is not currently running, but it is eligible to be selected by the scheduler. A process becomes Ready as soon as it is admitted, and it returns to Ready whenever the processor becomes free. The processes in the Ready state are organised into a ready queue, and the scheduler picks the next process to run from that ready queue.

A process in the Running state is the one process that currently holds the processor and is executing its instructions. At most one process per processor is in the Running state, so on a single core system exactly one process at a time is Running.

A process in the Waiting state, sometimes called the Blocked state, is waiting for an event that has not happened yet. A process usually enters it after requesting input or output, or while waiting for a lock, a signal, or another process to release a shared resource. A process in the Waiting state cannot be chosen by the scheduler until the awaited event occurs.

A process in the Terminated state has finished execution. The operating system keeps its process control block briefly so exit status can be reported, then reclaims the memory. A process in the Terminated state is never scheduled again, so a process enters it only once.

Process state transitions

A process moves from the New state to the Ready state once the operating system has finished admitting it. The common Ready to Running transition happens when the scheduler selects a process from the ready queue and gives it the processor.

A process moves from the Running state back to the Ready state when it gives up the processor without finishing. This happens in two important situations. The first is a preemption, in which a higher priority process becomes Ready and takes the processor away. The second is the expiry of a time slice, in which the operating system enforces

a fairness limit so that one process cannot hold the processor indefinitely.

A process moves from the Running state to the Waiting state when it cannot continue without an event, for example when it issues a disk read and must wait for the result.

A process moves from the Waiting state back to the Ready state when the event it was waiting for occurs, because it is then eligible to run again but may still have to wait for the processor. A process moves from the Running state directly to the Terminated state when its program reaches its normal exit point.

Context switching

A context switch is the operation of saving the execution context of one process and restoring the execution context of a different process. When the scheduler preempts a process, the operating system copies the register values and the program counter of the outgoing process into its process control block. It then loads the values from the process control block of the incoming process into the processor registers. The result is that the processor begins executing the incoming process as though it had been running all along.

A context switch happens whenever the scheduler decides to run a different process. The three common triggers are the expiry of a time slice, a running process blocking itself by entering the Waiting state, and a higher priority process becoming Ready.

A context switch has a real cost. While it runs, the processor executes no user work, so a system that switches too often spends much of its time saving and restoring state instead of computing, which is why very short CPU bursts are expensive to schedule.

Scheduling basics

The scheduler is the component of the operating system that decides which process in the Ready state should run next. A scheduler that always lets the current process finish before it makes any decision is called a non-preemptive scheduler. Under a non-preemptive scheduler, a process holds the processor until it terminates or until it blocks itself, so a long running process can delay every other process.

A scheduler that can interrupt a running process to give the processor away is called a preemptive scheduler. It is more responsive, because a process that becomes Waiting no longer holds the processor, but it pays a context switch each time a running process is displaced.

Three common policies illustrate the trade off. First Come First Served takes the process that has waited in the ready queue longest, and because it never preempts a single long process can produce a convoy effect. Shortest Job First selects the process with the shortest predicted CPU burst. Round Robin gives each process a fixed time slice in rotation, bounding every wait at the cost of frequent context switches.

A short worked example

Consider a program that reads a record from disk and then formats it. When the process is created it is in the New state, and after admission the operating system moves it to the Ready state. The scheduler eventually takes it from the ready queue and the process enters the Running state. The process then issues a disk read, but because the data is not in memory it cannot continue, so it moves from the Running state to the Waiting state and the operating system immediately schedules another process from the ready queue. That transition is one context switch. When the disk read completes, an interrupt moves the process from the Waiting state back to the Ready state, and the process runs again only after the scheduler selects it a second time. The second example is a compute-bound process that runs a long time without blocking. Under Round Robin it is displaced every time its time slice expires, moving from Running back to Ready, so it can be context switched many hundreds of times. Under a non-preemptive scheduler it keeps the processor until it terminates.